orcahand: API Reference for the MABEL ORCA Dexterous Hand

MABEL Robotics — ORCA Hand Subsystem

June 29, 2026

Abstract

`orcahand` is the single hardware-abstraction layer for the MABEL ORCA hand — a 17-actuator (16 finger + 1 wrist) tendon-driven hand on a 1 Mbps Feetech STS/SCS serial bus. It exposes one class, `OrcaHand`, that speaks SI units (radians, rad/s) and joint names, hiding tick conversion, calibration clamping, and serial framing. The same package is imported directly by controllers and learned policies, wrapped thinly by the ROS 2 node, and used by the desktop GUI (*ORCA Hand Studio*) and the command-line tools — replacing three previously-divergent driver implementations. This document is the detailed API reference: the layered architecture, the angle–tick safety map, a per-class reference, the stall-detection calibration routine, the CLI tools, the GUI, and worked examples (mock, hardware, controller, ROS).

Keywords: dexterous hand, Feetech servo, hardware abstraction, ROS 2, calibration, teleoperation, imitation learning.

1 Overview and layout

The ORCA hand previously had *three* parallel drivers: the firmware workbench scripts (with ∼5 ad-hoc `Bus` classes), a re-ported Feetech protocol inside the ROS node, and the upstream Dynamixel-based `orca_core`. They drifted: two unit conventions (percent vs. radians), duplicated joint maps, and incompatible calibration formats. `orcahand` unifies them into one importable package with a strict layering (Fig. 1): each layer depends only on the one below, so the wire protocol lives in exactly one place.

The whole hand stack now lives under `firmware/orca_hand/feetech/`: the importable package is `orcahand/` (so `import orcahand` is unchanged for ROS and the controllers), the GUI is `gui/`, the example programs are `control/`, and the CLI tools are `scripts/`. Table 1 summarises the tree.

2 Method: angle–tick mapping and safety

The hand is commanded in joint angles but the servos accept 12-bit encoder *ticks* (0 . . . 4095, 4096 ticks/rev). Each joint j carries a logical range $[\theta_j^{\min}, \theta_j^{\max}]$ (radians, from the URDF) and a *measured* tick range $[r_j^{\min}, r_j^{\max}]$ (the calibrated hardware end-stops). The command map is the affine fit between these intervals. Writing $(\rho_0, \rho_1) = (r_j^{\min}, r_j^{\max})$, swapped to $(r_j^{\max}, r_j^{\min})$ when the joint is mechanically inverted,

$$t = \mathrm{clip}\left(\frac{\theta - \theta_j^{\min}}{\theta_j^{\max} - \theta_j^{\min}},\, 0,\, 1\right), \qquad r(\theta) = \mathrm{clip}\big(\rho_0 + t\,(\rho_1 - \rho_0),\, r_j^{\min},\, r_j^{\max}\big), \tag{1}$$

Table 1: Repository layout under `firmware/orca_hand/feetech/`.

Path	Contents
`orcahand/`	the package: `hand.py`, `bus.py`, `config.py`, `joints.py`, `calibration.py`, `calibrate.py`, `state.py`, `poses.py`, `ports.py`, `mock.py`, `cli.py`, `configs/`
`scripts/`	CLI tools: `scan`, `bringup`, `tension`, `calibrate`, `poses`, `slider`
`gui/`	*ORCA Hand Studio* (Tkinter) + the `HandBus` adapter
`control/`	example/demo programs
`tests/`	12 hardware-free contract tests
`models/`, `assets/`	MuJoCo simulation assets (unrelated to the driver)

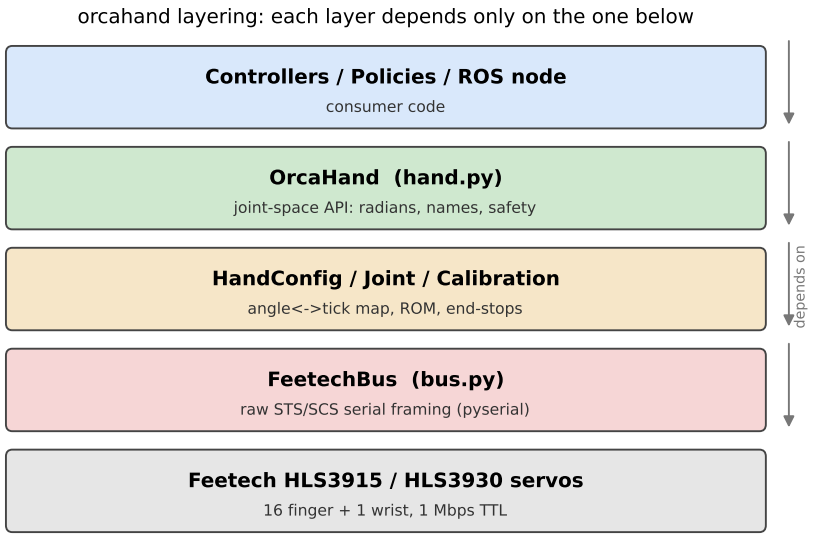


Figure 1: The `orcahand` stack. Controllers, policies, the ROS node, the GUI, and the demos all enter at `OrcaHand`; only `FeetechBus` touches the serial wire.

and the inverse, used for feedback, is

$$\theta(r) = \theta_j^{\min} + \frac{r - \rho_0}{\rho_1 - \rho_0}\left(\theta_j^{\max} - \theta_j^{\min}\right). \tag{2}$$

The double clip in (1) is the safety guarantee: the normalised t is clamped to $[0, 1]$ *and* the tick is clamped to the calibrated span, so no commanded angle — however out of range — can drive a tendon past its physical stop. Velocity feedback uses $\dot{\theta} = s \cdot (2\pi/4096)$ for raw speed s (sign-flipped if inverted); effort is the normalised servo load $\ell/1000 \in [-1, 1]$. The percent helpers use a separate, inversion-agnostic scale $p \in [0, 100]$ with $r(p) = r_j^{\min} + \frac{p}{100}(r_j^{\max} - r_j^{\min})$. Fig. 2 plots (1) for a real joint, with and without inversion.

2.1 Single-turn wrap and the software offset

The STS/SCS encoder is *single-turn absolute*: it reports $0 \ldots 4095$ and rolls straight over $(4095 \to 0)$. A joint whose travel happens to straddle that $0/4095$ seam reads discontinuously, and the affine map of §2 — which assumes a monotone $[\rho_0, \rho_1]$ — breaks. Because every ORCA tendon joint

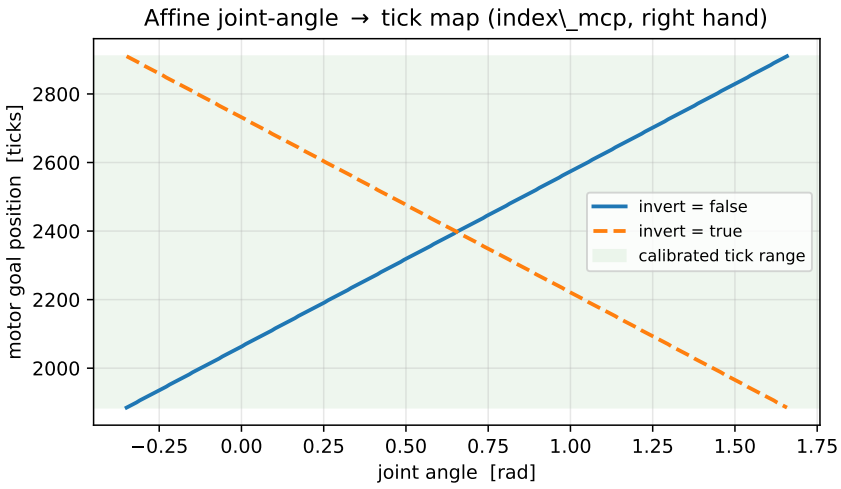


Figure 2: Eq. (1) for `index_mcp`, using its measured calibration. `invert` swaps which tick end-stop maps to the maximum angle; the shaded band is the hard tick clamp.

sweeps well under one full turn ($s < 4096$ ticks), the seam can always be relocated in *software*: each joint carries an integer *offset* o_j and the map of §2 is evaluated on the *corrected* tick

$$c = (r - o_j) \bmod 4096, \qquad r = (c + o_j) \bmod 4096, \tag{3}$$

with $r_j^{\min}, r_j^{\max}$, and the centre all stored in the corrected frame. The command map re-applies the right-hand side of (3) so the value written to the servo is always a real tick in $[0, 4095]$. For the common case (the calibrated arc already fits in-window without crossing the seam) $o_j = 0$ and (3) is the identity — the mapping, the stored numbers, and the file are byte-identical to the offset-free layer, so only a genuine seam-straddler is ever affected.

The offset is chosen at calibration time. A by-hand sweep is tracked in a *continuous* (unwrapped) coordinate by integrating the shortest signed step between successive reads,

$$u_k = u_{k-1} + \delta(r_k - r_{k-1}), \qquad \delta(d) = \big((d + 2048) \bmod 4096\big) - 2048, \tag{4}$$

which is exact as long as the joint moves < 2048 ticks between samples (always true at the poll rate). Given the swept arc $[u_{\min}, u_{\max}]$ with span $s = u_{\max} - u_{\min}$, if $(u_{\min} \bmod 4096) + s \le 4095$ the arc fits in-window and $o_j = 0$; otherwise the seam is relocated so the arc centre maps to corrected 2048,

$$o_j = \Big(\big\lfloor \tfrac{u_{\min} + u_{\max}}{2} \big\rfloor \bmod 4096 - 2048\Big) \bmod 4096, \qquad [r_j^{\min}, r_j^{\max}] = \big[2048 - \tfrac{s}{2},\, 2048 + \tfrac{s}{2}\big]. \tag{5}$$

This is computed entirely inside the hardware layer (the GUI's manual calibration); the calibration file gains an optional `"offset"` key (default 0), and ROS still receives only calibrated radians via (2) — it never sees ticks or offsets. A *powered* stall sweep cannot itself cross the seam (its goal clamps at 0/4095), so a joint that stalls there is flagged for a hand sweep — which auto-wraps via (4)–(5) — or for a hardware re-center that burns the encoder zero and resets $o_j = 0$.

3 Installation

The layer depends only on `pyserial`, `pyyaml`, and `numpy` — no vendor SDK — so it installs into any Python ≥ 3.9 environment, including the one ROS uses.

Table 2: `OrcaHand` public methods.

Call	Effect
`connect(enable_torque=True, scan=True)`	open bus, push motion defaults, scan, torque on
`disconnect()`	disable torque, close bus (idempotent)
`set_joint_positions(targets)`	command angles [rad]; dict (partial) or full array
`get_joint_positions(as_array=False)`	present angles [rad] by name, or ordered list
`get_state()`	`HandState`: position/velocity/effort/temperature/-voltage/raw
`set_joint_percent({name: pct})`	0–100 % of range (0 = min tick, 100 = max tick)
`set_named_position(name)`	drive to a named pose (`hand.pose_names`)
`enable_torque(joints=None)` / `disable_torque(...)`	all motors, or a named subset
`joint_names`, `motor_ids`, `missing_joints`	introspection (canonical order)
`is_connected`, `is_torque_enabled`, `side`, `bus`	state / handles
`OrcaHand.mock(model)`	in-memory bus, identical API, no hardware

```
cd firmware/orca_hand/feetech
pip install -e .          # editable; installs the orcahand-scan/-calibrate CLIs
```

Verify with the bundled hardware-free tests: `python3 -m pytest tests/` (12 contract tests; no hardware required).

4 API contract

A single class, `OrcaHand`, is the primary public surface. Construct with a model name (`"left"`/`"right"`, loading the packaged canonical config) or explicit `config_path`/`calibration_path`. Table 2 lists the methods.

- **Units.** Angles are radians, velocities rad/s, effort normalised $[-1, 1]$, temperature °C, voltage V.
- **Inputs.** A {`joint_name: angle`} dict (*partial is fine* — only named joints move) or a full array ordered by `hand.joint_names`.
- **Lifecycle.** `connect`/`disconnect` are idempotent; `disconnect` always disables torque first. `OrcaHand` is a context manager and thread-safe (one internal bus lock), so a feedback thread and a command thread can share one hand.
- **Safety.** Every command is clamped per (1). Motors that fail the connect-time scan are dropped from the active set and reported via `hand.missing_joints`; the hand still runs on whatever responds.

5 Class reference

The package layers five small classes (Fig. 1). Each is documented below; together they are the complete API.

5.1 `OrcaHand` (`hand.py`)

The joint-space interface. Constructor: `OrcaHand(model=None, *, config_path=None, calibration_path=None, port=None, baudrate=1 000 000, torque_limit=500, goal_speed=2000, goal_acc=40, bus=None)`. `model` loads the packaged config; `port=None` auto-detects the USB adapter; `bus` injects a custom transport (used by `mock`). Motion defaults (`torque_limit`, `goal_speed`, `goal_acc`) are written to every motor on `connect`. Public methods are in Table 2.

5.2 `HandConfig` (`config.py`)

The static description of one hand: an ordered list of `Joint` objects plus the resolved config/calibration paths. Load with `HandConfig.load(model=None, config_path=None, calibration_path=None)`. Lookups: `joint_names`, `motor_ids`, `joint(name)`, `by_name()`→{`name:Joint`}, `by_id()`→{`id:Joint`}; `HandConfig.available_models()` lists the packaged configs. The YAML schema is in §10.

5.3 `Joint` (`joints.py`)

Couples a joint's logical description (`name`, `angle_min`, `angle_max`, `invert`) with its measured calibration (`raw_min`, `raw_max`, `center`) and the single-turn `offset` of §2.1 (default 0). Implements the maps of §2, evaluated on the corrected tick of Eq. (3): `angle_to_raw(`θ`)` / `raw_to_angle(r)` (Eqs. 1–2), `percent_to_raw(p)` / `raw_to_percent(r)`, `speed_to_rad_s`, `load_to_effort`, and `clamp_angle`. Properties `raw_lo`, `raw_hi` give the unordered tick span used for clamping.

5.4 `Calibration` / `MotorCalibration` (`calibration.py`)

The measured tick end-stops, keyed by motor ID — the hard safety clamp. `Calibration.from_file(path)` reads the JSON; `get(id)` returns a `MotorCalibration(motor_id, raw_min, raw_max, center, offset=0)` (with a full $0 \ldots 4095$ fallback for any uncalibrated motor) and `.span` gives $r^{\max} - r^{\min}$. The optional `offset` (§2.1) defaults to 0 so legacy files load unchanged. File format (the same the firmware emits; the `"offset"` key is written only when non-zero matters):

```
{ "7": { "joint": "index_mcp", "min": 1885, "max": 2910,
         "center": 2397, "range": 1025 }, ... }
```

5.5 `HandState` (`state.py`)

An immutable, thread-safe snapshot returned by `get_state()`: dicts keyed by joint name — `position` [rad], `velocity` [rad/s], `effort` $[-1, 1]$, `temperature` [°C], `voltage` [V], `raw_position` [ticks] — plus a `stamp`. Helpers: `positions_array(names)` and `hottest()`→`(joint,` °C`)`.

5.6 `FeetechBus` (`bus.py`)

The only code that touches the wire: a dependency-free implementation of the Feetech 1.0 packet protocol over `pyserial`. It deals purely in motor IDs and raw hardware units; all joint-name/radian logic lives above it. Opening takes a POSIX exclusive lock so two processes cannot trample one adapter. Table 3 lists its surface; controllers rarely call it directly, but the GUI and the calibration routine do.

Table 3: `FeetechBus` low-level methods (raw ticks / motor IDs).

Call	Effect
`open()` / `close()` / `is_open`	lifecycle (exclusive serial lock)
`ping(id)` / `scan(range)`	discovery
`sync_write_positions({id: raw})`	one SYNC_WRITE goal-position packet (hot path)
`read_positions(ids)`	fast SYNC_READ of positions (per-motor fallback)
`read_feedback(id)`	8-byte block: position, speed, load, voltage, temp
`read_register(id, addr, len)`	generic control-table read
`write_register(id, addr, data)`	generic control-table write
`set_torque_enable(on, id)`	torque on/off (broadcast by default)
`set_torque_limit` / `set_goal_speed` / `set_goal_acc`	motion-register config

5.7 Helpers: `poses.py`, `ports.py`, `mock.py`

`poses.POSES` is the named-gesture table (percent per joint), consumed by `set_named_position`. `ports.detect_port()` is the single USB-adapter auto-detector (CH340 / CP210x by vendor ID, then description, then sole port). `mock.MockFeetechBus` is the in-memory transport behind `OrcaHand.mock` — identical surface, no serial I/O.

6 Calibration routine (`orcahand.calibrate`)

Calibration measures each joint's tick end-stops $[r_j^{\min}, r_j^{\max}]$ that drive (1). The routine is the Feetech analogue of `orca_core`'s current-based-position sweep: the Feetech servo has no current-position mode but *does* have a torque ceiling, so the algorithm caps torque low and walks the goal $r \leftarrow r_{\text{present}} + d \cdot \Delta$ (direction $d = \pm 1$, step Δ) each tick. While the joint is free the encoder follows; once it jams against the end-stop the limited torque cannot push further, the encoder freezes, and a rolling-window stability test fires:

$$\text{stop detected} \iff \max_k(w_k) - \min_k(w_k) \le \tau \ \text{ for the last } N \text{ samples}, \tag{6}$$

with window w of the last N encoder reads and threshold τ ticks (after a short warm-up so motion can begin). Both directions give $r^{\min}$, $r^{\max}$; the centre is $\lfloor (r^{\min} + r^{\max})/2 \rfloor$ and the gear ratio for each joint follows from its span over the URDF range. Table 4 lists the tuning knobs (`CalibrationParams`).

Table 4: `CalibrationParams` (Feetech analogues of orca_core's knobs).

Field	Default	Meaning
`calib_torque`	220	torque ceiling during the sweep (0–1000)
`goal_speed`	600	ticks/s cap (slow, controlled)
`step_size`	18	ticks added toward the stop per iteration
`step_period`	20 ms	between goal nudges / encoder reads
`num_stable` (N)	8	consecutive in-window samples ⇒ stalled
`stable_threshold` (τ)	6	max tick spread of the window
`min_span` / `max_span`	120 / 3600	sanity bounds (barely-moved / wrap)

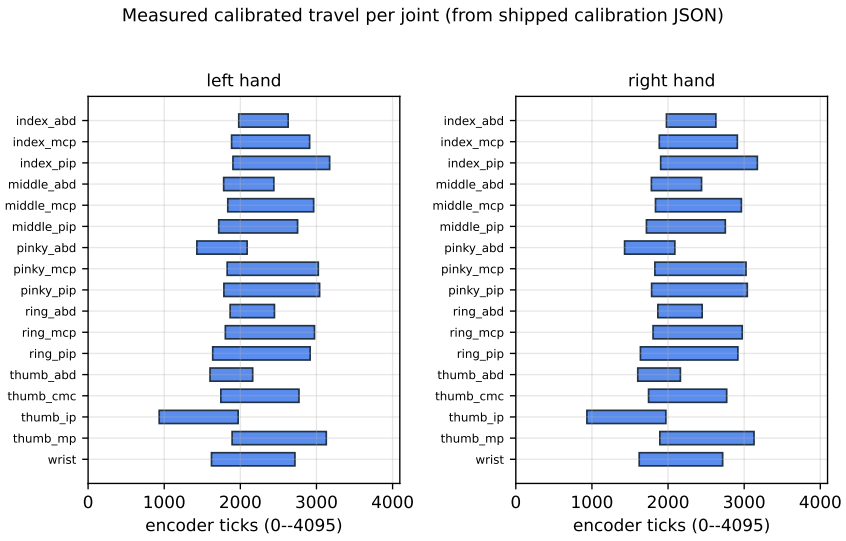


Figure 3: Calibrated tick ranges $[r_j^{\min}, r_j^{\max}]$ per joint (left hand), from `configs/left_calibration.json`; wider spans give finer angular resolution.

Usage. The routine is exposed as a module, a console script, and a `scripts/` wrapper:

```
python3 -m orcahand.calibrate --model left           # = orcahand-calibrate
python3 scripts/calibrate.py   --model left --joints index_mcp index_pip
python3 scripts/calibrate.py   --model left --sim     # hardware-free self-test
```

It writes the `<side>_calibration.json` of §5 (backing up the previous file), parks each joint at its measured centre, and releases torque on exit (including Ctrl-C). The same sweep is also exposed as the ROS `hand_calibration_node`.

Verification. The `--sim` mode seeds a synthetic end-stop bus from the current calibration and runs the *real* routine against it. On the MABEL left hand it recovers all 17 end-stops at **0-tick error** (mean and max), confirming the stall detector and the motor→joint mapping. Fig. 3 shows the calibrated tick spans per joint.

7 Command-line tools (`scripts/`)

All are built on the API above; each takes `--model left|right`, `--port`, and `--mock`. Table 5 lists them.

Table 5: `scripts/` CLI tools.

Command	Purpose
`scan.py`	ping the bus, list responding motors by joint
`bringup.py`	live telemetry dashboard (`--watch`); optional `--pose`
`tension.py`	gentle oscillation to seat tendons before calibration
`calibrate.py`	end-stop calibration (§6)
`poses.py`	cycle named gestures
`slider.py`	Tkinter per-joint slider panel

8 GUI: ORCA Hand Studio (gui/)

The headline interface is a Tkinter studio with a live telemetry sidebar and Control / Poses / Teleop / Calibration / Setup panels. It runs entirely on the `orcahand` API through a thin adapter, `HandBus`, which owns an `OrcaHand`, polls `get_state()` at 10 Hz, and exposes the motor-ID/raw-tick surface the panels expect (`connect`, `sync_write`, `set_torque`, `pct_to_raw`, `read_positions_direct`, edited calibration save/reload, etc.). Because every panel goes through the same API as the ROS node and the controllers, the GUI can never drift from them.

```
python3 gui/gui.py                  # prompts for port + hand
python3 gui/gui.py --hand left      # preselect a hand
python3 gui/gui.py --mock           # full GUI, no hardware
```

The example programs in `control/` (e.g. `demo_gestures.py`, `torque_off.py`) draw their config, poses, and port detection from the API via the `utils/hand_config.py` compatibility shim, whose `connect_hand()` returns a ready `OrcaHand`.

9 Usage

Hardware-free (mock). Develop and test logic with no hand attached; the mock round-trips `set`→`get` exactly (minus dynamics).

```
from orcahand import OrcaHand

hand = OrcaHand.mock("left")           # in-memory bus
hand.connect()
hand.set_joint_positions({"index_mcp": 0.5, "thumb_cmc": 0.1})  # radians
print(hand.get_joint_positions()["index_mcp"])   # ~0.50
hand.disconnect()
```

Real hand. The `with` block connects (torque on) and, on exit, disables torque and closes the port — even on exception.

```
from orcahand import OrcaHand

with OrcaHand("left") as hand:         # USB adapter auto-detected
    hand.set_named_position("flat")
    hand.set_joint_positions({"index_mcp": 0.6, "middle_mcp": 0.6})
    state = hand.get_state()
    print(state.position["index_mcp"], state.hottest())
```

Pin the port when both hands are attached (auto-detect cannot tell two identical adapters apart): `OrcaHand("left", port="/dev/ttyUSB0")`.

Controller / policy. Align the policy output vector with `hand.joint_names`, the canonical ordering.

```
class HandController:
    def __init__(self, side="left"):
        self.hand = OrcaHand(side).connect()
        self.order = self.hand.joint_names        # 17 names
```

```
    def step(self, q_des):                          # q_des: np.ndarray, len 17
        self.hand.set_joint_positions(q_des.tolist())
        return self.hand.get_joint_positions(as_array=True)
```

ROS 2. The `mabel_hand_hardware` package wraps `orcahand`; one node per hand. It subscribes `command` (`sensor_msgs/JointState`, radians) and publishes `joint_states` (plus temperature/voltage), and ships a `hand_calibration_node` sharing the sweep of §6.

```
pip install -e firmware/orca_hand/feetech       # into the ROS python env
cd mabel_ws && colcon build --packages-select mabel_hand_hardware --symlink-install
```

Command line. `orcahand-scan --model left` pings the bus and lists which motor IDs answer, annotated with joint names — the fastest health check after wiring.

10 Configuration

A hand is one YAML file plus a calibration JSON. Canonical MABEL configs ship in `orcahand/configs/`, so `OrcaHand("left")` works out of the box; pass `config_path`/`calibration_path` to use project-specific files (the ROS node feeds its `share/config` paths this way). The YAML gives the joint→ID map, the angle range, and the inversion flag; the JSON (§5) — produced by `orcahand.calibrate` — gives the measured tick end-stops that drive (1).

```
side: left
calibration_file: left_calibration.json    # relative to this yaml, or absolute
joints:
  index_mcp: { id: 7, angle_min: -0.3491, angle_max: 1.6581, invert: false }
  # ... 17 entries (16 finger + wrist). NB: the left/right ID maps differ.
```

11 Limitations

- **Position control only.** The API commands goal positions; speed and acceleration are set once at connect (servo on-board S-curve does the smoothing). There is no torque/current command path yet, and no on-line trajectory blending beyond what the servo firmware provides.
- **Open-loop angle accuracy.** (1) is linear in ticks; tendon non-linearity, coupling, and hysteresis are not modelled, so commanded vs. realised joint angle can disagree near the extremes. The map is only as good as the calibration JSON.
- **Feedback rate.** `get_state()` issues one read per active motor; at 17 motors this caps the practical feedback loop around 50 Hz to 100 Hz. `get_joint_positions()` uses a faster position-only sync read with a per-motor fallback.
- **No tactile.** Upstream `orca_core` has a tactile path; this layer is motors only.

References

[1] ORCA Hand, *orca_core control library*, https://github.com/orcahand/orca_core (upstream reference for the layered API and the current-based-position calibration sweep).

[2] FEETECH, *STS/SCS Serial Bus Servo Communication Protocol* (1 Mbps TTL half-duplex, 12-bit position feedback).

[3] MABEL Robotics, *ORCA Hand Firmware: Bus, Setup, and Calibration* (companion document, `docs/orca_firmware.pdf`).